

Claude Codeで作るネイティブコンパイラ

2026年05月21日（木）
株式会社ネクストビート

本日の流れ

時間	内容
19:00-19:10	受付・自己紹介
19:10-19:20	イントロ：Claude Code × ネイティブコンパイラ
19:20-20:20	ハンズオン（60分）
20:20-20:50	お寿司＋振り返り・Q&A
20:50-21:00	片付け・撤収

会社紹介

イントロ：Claude Code × ネイティブコンパイラ

「Claude Codeで色々なソフトウェアを作れる」——聞き飽きた頃ではないでしょうか。

今日のテーマは、一段踏み込んで：

- **Claude Code / Codex** を相棒に、小さな言語のネイティブコンパイラを書く
- 生成された artifacts から実バイナリが生まれ、`./a.out` で動く
- エージェント時代のコンパイラ開発の感触を実地で確かめる

なぜ「ネイティブコンパイラ」なのか

- インタプリタは **AST** を走査しながら実行する
- コンパイラは **AST** → 中間表現 → ネイティブコード に変換する
- 後者には **型・レジスタ・ABI・リンク** など別種の難しさがある

エージェント時代、この「別種の難しさ」をどこまで預けられるか？

今日のゴール

1. 小さな言語 "nb-lang" のネイティブコンパイラを書く

- 実装言語は **Scala 3 (sbt) / TypeScript (Bun) / Java (Maven)** から好きなものを
- 整数演算 / 変数 / if / while / print
- 出力：LLVM IR → clang でリンク → 実バイナリ

2. Claude Code / Codex との協業設計を体験する

- 丸投げする部分と、人間が指揮する部分の切り分け

3. 動くバイナリでなんかする

- `./a.out` を叩いて結果を見る達成感

4. (Optional) 言語拡張チャレンジ

バックエンド戦略

基本線：LLVM IR を文字列として吐く → `llc` でオブジェクトコード → `clang` でリンク

代替：

- **C** を吐いて `gcc/clang` に任せる（一番ラク）
- 生アセンブリを直接吐く（手触りはあるが 1 時間では厳しい）

ハンズオン開始：リポジトリ準備

まずはリポジトリを clone してください。プロンプトとリファレンス実装が入っています。

```
git clone git@github.com:nextbeat-dev/nextbeat-tech-event.git
cd nextbeat-tech-event/cc-native-compiler-osaka-202605
```

ブラウザで中身を確認したい人はこちら：

<https://github.com/nextbeat-dev/nextbeat-tech-event/tree/main/cc-native-compiler-osaka-202605>

ハンズオン開始：作業ディレクトリの用意

clone したリポジトリは「配布物」。書き換えるのではなく、別場所に作業ディレクトリを切ります：

```
mkdir -p ~/work/nblang && cd ~/work/nblang
claude      # Claude Code
# または
codex       # Codex CLI
```

エージェント CLI は **Claude Code / Codex** のどちらでも **OK**。起動したら、次ページへ：

プロンプトファイルの参照

自分の言語に応じて以下のプロンプトファイルを参照：

言語	参照するファイル
Scala 3	~/.../cc-native-compiler-osaka-202605/prompts/scala3.md
TypeScript	~/.../cc-native-compiler-osaka-202605/prompts/typescript.md

ハンズオン（前半 30分）

Step 1: AST と字句・構文解析

Claude Code もしくは Codex を起動して、自分の言語のプロンプト 1 ファイルを `@` で渡してスタート。

```
@~/.../cc-native-compiler-osaka-202605/prompts/scala3.md
```

このプロンプトに従って、nb-lang のネイティブコンパイラを Phase 1 から作ってください。
最初のゴールは examples/sum.nb が 55 を出すところまで。

困ったときの参考実装は `langs/{scala3, typescript, java}/` 配下にあります
（前者は配布リポジトリ、ここを覗くのはカンニング OK ということで）。

ハンズオン（後半 30分）：言語拡張チャレンジ

前半で Phase 1～3 が動いたら、後半は **自分のコンパイラに機能を足す** ターン。
4 つの拡張プロンプトを `prompts/ext-*.md` に用意しています。

拡張	プロンプト	触る場所	達成感
<code>break</code> / <code>continue</code>	<code>prompts/ext-break-continue.md</code>	Lexer/Parser/AST/CodeGen	basic block の感触◎
<code>&&</code> / <code> </code> / <code>!</code>	<code>prompts/ext-logical-ops.md</code>	Lexer/Parser/AST/CodeGen	短絡評価を IR で表現
C 風 <code>for</code>	<code>prompts/ext-for-loop.md</code>	Lexer/Parser (+ CodeGen)	while への desugar 体験
<code>string + string</code>	<code>prompts/ext-string-concat.md</code>	TypeCheck/CodeGen	外部ランタイム関数の呼び出し

好きなのを **1 個選んで**ぶん投げ。複数いけたらボーナス。
株式会社ネクストビート

言語拡張チャレンジ：投げ方の例

Claude Code / Codex に **自分の言語の prompt + 拡張 prompt** を一緒に渡す：

```
@~/.../cc-native-compiler-osaka-202605/prompts/scala3.md  
@~/.../cc-native-compiler-osaka-202605/prompts/ext-logical-ops.md
```

このプロンプトに従って、既存の実装に `&&` / `||` / `!` を短絡評価で追加してください。
追加後、`examples/ext_logical.nb` と `ext_short_circuit.nb` で動作を確認してください。

各 `ext-*.md` には **仕様 / 各層の変更点 / テストプログラム / 期待出力 / プロンプト雛形** が全部入っているので、Claude Code / Codex は迷いません。

困ったら参考実装 `langs/<言語>/` を覗いて差分を相談するのも有効。

まとめ

- Claude Code / Codex で **1 時間あればネイティブコンパイラは書ける**
- 肝は「**仕様と戦略を人間が決める、実装は預ける**」
- 型と ABI の境界で人間の介入が必要になる
- 今日のコードは各自の GitHub で育ててください

質疑応答・お寿司タイム

ご清聴ありがとうございました。

質問・感想・実装の疑問、お気軽にどうぞ。